

SkyPredict: Flight Price Analysis and Delay Prediction System



A Big Data Analytics & Machine Learning Project

Python | Pandas | Seaborn | Matplotlib | Scikit-learn | PySpark

Dataset: Airlines Flights Data — 300,153 records
17 June 2026

Project Description

Smart Aviation Analytics: Flight Price Analysis and Delay Prediction Using Python, Big Data Analytics, and Machine Learning

This project focuses on analyzing airline flight data to understand ticket pricing patterns and predict potential flight delays using data analytics and machine learning techniques. The aviation industry generates large volumes of data daily, making it an ideal domain for applying Big Data Analytics to derive meaningful insights and support decision-making.

The dataset used in this project contains over 300,000 flight records from various airlines operating between major Indian cities. It includes information such as airline name, source and destination cities, departure and arrival times, number of stops, travel class, flight duration, days left before departure, and ticket price. The data is cleaned, processed, and analyzed using Python libraries including Pandas, NumPy, Matplotlib, and Seaborn.

The first phase of the project focuses on exploratory data analysis to identify factors affecting ticket prices. Various analyses are performed to compare airline pricing strategies, evaluate the impact of travel class, study route-based fare variations, analyze departure and arrival schedules, and understand how booking tickets closer to the departure date influences prices. Statistical summaries and visualizations are used to uncover trends and business insights.

The second phase introduces a Flight Delay Prediction system. Using historical flight information and machine learning algorithms, the project aims to predict whether a flight is likely to be delayed based on factors such as airline, route, departure schedule, flight duration, weather conditions, and operational patterns. Such predictions can help airlines improve resource management and enable passengers to make informed travel decisions.

Overall, the project demonstrates the practical application of Big Data Analytics, Business Intelligence, Data Visualization, and Machine Learning in the aviation sector. The outcomes provide valuable insights into airline operations, pricing behavior, and travel trends while laying the foundation for intelligent aviation solutions such as real-time delay prediction, AI-powered travel assistants, and interactive analytics dashboards. This project highlights how data-driven technologies can improve operational efficiency, customer satisfaction, and strategic decision-making in the airline industry.

Step 1 — Import Libraries & Load Dataset

Import all required Python libraries and load the Airlines Flights dataset into a Pandas DataFrame.

```
# import the required Python libraries

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline
import seaborn as sns
```

```
# import the Flights dataset

data = pd.read_csv(r"C:\Users\Roshiga\Downloads\airlines_flights_data.csv")
```

	index	airline	flight	source_city	departure_time	stops	arrival_time	destination_city
0	0	SpiceJet	SG-8709	Delhi	Evening	zero	Night	Mumbai
1	1	SpiceJet	SG-8157	Delhi	Early_Morning	zero	Morning	Mumbai
2	2	AirAsia	I5-764	Delhi	Early_Morning	zero	Early_Morning	Mumbai
3	3	Vistara	UK-995	Delhi	Morning	zero	Afternoon	Mumbai
4	4	Vistara	UK-963	Delhi	Morning	zero	Morning	Mumbai
...	...	...	...	...	...	...	...	...
300148	300148	Vistara	UK-822	Chennai	Morning	one	Evening	Hyderabad
300149	300149	Vistara	UK-826	Chennai	Afternoon	one	Night	Hyderabad
300150	300150	Vistara	UK-832	Chennai	Early_Morning	one	Night	Hyderabad
300151	300151	Vistara	UK-828	Chennai	Early_Morning	one	Evening	Hyderabad
300152	300152	Vistara	UK-822	Chennai	Morning	one	Evening	Hyderabad

Step 2 — Cleaning the Data

Remove the index column

```
# Remove the 'index' column

data.drop( columns = 'index', inplace = True)
data
```

	airline	flight	source_city	departure_time	stops	arrival_time	destination_city	class
0	SpiceJet	SG-8709	Delhi	Evening	zero	Night	Mumbai	Economy
1	SpiceJet	SG-8157	Delhi	Early_Morning	zero	Morning	Mumbai	Economy
2	AirAsia	I5-764	Delhi	Early_Morning	zero	Early_Morning	Mumbai	Economy
3	Vistara	UK-995	Delhi	Morning	zero	Afternoon	Mumbai	Economy
4	Vistara	UK-963	Delhi	Morning	zero	Morning	Mumbai	Economy
...	...	...	...	...	...	...	...	...
300148	Vistara	UK-822	Chennai	Morning	one	Evening	Hyderabad	Business
300149	Vistara	UK-826	Chennai	Afternoon	one	Night	Hyderabad	Business
300150	Vistara	UK-832	Chennai	Early_Morning	one	Night	Hyderabad	Business
300151	Vistara	UK-828	Chennai	Early_Morning	one	Evening	Hyderabad	Business
300152	Vistara	UK-822	Chennai	Morning	one	Evening	Hyderabad	Business

300153 rows × 11 columns

Dataset Info

```
# Get some Info about the dataset
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 300153 entries, 0 to 300152
Data columns (total 11 columns):
#   Column                Non-Null Count  Dtype
---  -
0   airline                300153 non-null object
1   flight                 300153 non-null object
2   source_city            300153 non-null object
3   departure_time         300153 non-null object
4   stops                 300153 non-null object
5   arrival_time           300153 non-null object
6   destination_city       300153 non-null object
7   class                  300153 non-null object
8   duration                300153 non-null float64
9   days_left              300153 non-null int64
10  price                  300153 non-null int64
dtypes: float64(1), int64(2), object(8)
memory usage: 25.2+ MB
```

```
[16]:
```

Statistical Summary

```
# Get Statistical summary about the dataset

data.describe()
```

```
data.describe()
```

```
[16]:
```

	duration	days_left	price
count	300153.000000	300153.000000	300153.000000
mean	12.221021	26.004751	20889.660523
std	7.191997	13.561004	22697.767366
min	0.830000	1.000000	1105.000000
25%	6.830000	15.000000	4783.000000
50%	11.250000	26.000000	7425.000000
75%	16.170000	38.000000	42521.000000
max	49.830000	49.000000	123071.000000

Check Extreme Duration Values

```
data[ data['duration'] == 49.830000 ]
```

```
[17]:
```

	airline	flight	source_city	departure_time	stops	arrival_time	destination_city	class	duration
193889	Air_India	AI-672	Chennai	Evening	two_or_more	Evening	Bangalore	Economy	0.83
194359	Air_India	AI-672	Chennai	Evening	one	Evening	Bangalore	Economy	0.83

```
data[ data['duration'] == 0.830000 ]
```

```
[18]: data[ data['duration'] == 0.830000 ]
```

```
[18]:
```

	airline	flight	source_city	departure_time	stops	arrival_time	destination_city	class	duration	days_left	price
115869	Indigo	6E-357	Bangalore	Night	zero	Night	Chennai	Economy	0.83	6	3498
115943	Indigo	6E-357	Bangalore	Night	zero	Night	Chennai	Economy	0.83	7	3498
116010	Indigo	6E-357	Bangalore	Night	zero	Night	Chennai	Economy	0.83	8	3498
116081	Indigo	6E-357	Bangalore	Night	zero	Night	Chennai	Economy	0.83	9	3498
116163	Indigo	6E-357	Bangalore	Night	zero	Night	Chennai	Economy	0.83	10	3498
116236	Indigo	6E-357	Bangalore	Night	zero	Night	Chennai	Economy	0.83	11	3498
116322	Indigo	6E-357	Bangalore	Night	zero	Night	Chennai	Economy	0.83	12	3498
116411	Indigo	6E-357	Bangalore	Night	zero	Night	Chennai	Economy	0.83	13	3498
116496	Indigo	6E-357	Bangalore	Night	zero	Night	Chennai	Economy	0.83	14	3498
116656	Indigo	6E-357	Bangalore	Night	zero	Night	Chennai	Economy	0.83	16	1924
116835	Indigo	6E-357	Bangalore	Night	zero	Night	Chennai	Economy	0.83	18	1924
116924	Indigo	6E-357	Bangalore	Night	zero	Night	Chennai	Economy	0.83	19	1924
117019	Indigo	6E-357	Bangalore	Night	zero	Night	Chennai	Economy	0.83	20	1924
117101	Indigo	6E-357	Bangalore	Night	zero	Night	Chennai	Economy	0.83	21	1924
117100	Indigo	6E-357	Bangalore	Night	zero	Night	Chennai	Economy	0.83	22	1924

Check Extreme Price Values

```
data [ data['price'] == 123071.000000 ]
```

```
[19]: data [ data['price'] == 123071.000000 ]
```

```
[19]:
```

	airline	flight	source_city	departure_time	stops	arrival_time	destination_city	class	duration	days_left	price
261377	Vistara	UK-772	Kolkata	Morning	one	Night	Delhi	Business	13.5	3	123071

Check for Missing Values

```
data.isnull().sum() # check for the missing values in any column
```

```
[20]: airline      0
      flight      0
      source_city  0
      departure_time 0
      stops       0
      arrival_time 0
      destination_city 0
      class       0
      duration    0
      days_left   0
      price       0
      dtype: int64
```

Q.1 What are the airlines in the dataset, accompanied by their frequencies?

View first few rows

```
data.head()
```

	airline	flight	source_city	departure_time	stops	arrival_time	destination_city	class	duration	days_left	price
0	SpiceJet	SG-8709	Delhi	Evening	zero	Night	Mumbai	Economy	2.17	1	5953
1	SpiceJet	SG-8157	Delhi	Early_Morning	zero	Morning	Mumbai	Economy	2.33	1	5953
2	AirAsia	I5-764	Delhi	Early_Morning	zero	Early_Morning	Mumbai	Economy	2.17	1	5956
3	Vistara	UK-995	Delhi	Morning	zero	Afternoon	Mumbai	Economy	2.25	1	5955
4	Vistara	UK-963	Delhi	Morning	zero	Morning	Mumbai	Economy	2.33	1	5955

Count unique airlines

```
# Checking how many Airlines are in the dataset
data['airline'].nunique()
```

```
[22]: 6
```

Get airline names

```
# Showing the names of the Airlines in the dataset
data['airline'].unique()
```

```
array(['SpiceJet', 'AirAsia', 'Vistara', 'GO_FIRST', 'Indigo',
       'Air_India'], dtype=object)
```

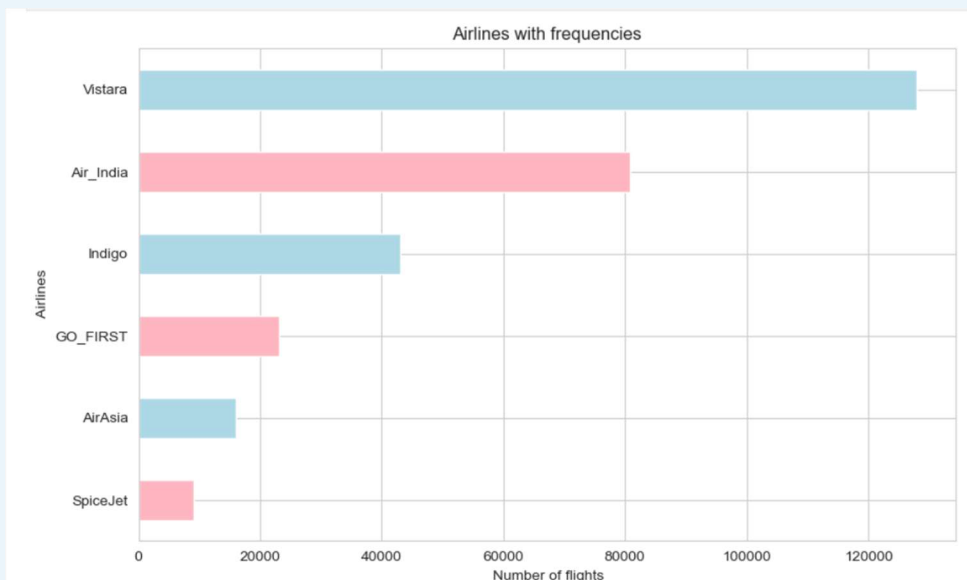
Airline frequencies

```
# Showing all the Airlines with their frequencies  
  
data['airline'].value_counts()
```

```
airline  
Vistara      127859  
Air_India    80892  
Indigo       43120  
GO_FIRST    23173  
AirAsia      16098  
SpiceJet     9011  
Name: count, dtype: int64
```

Horizontal Bar Graph — Airlines with frequencies

```
# Showing all the Airlines with their Number of Flights in Horizontal Bar Graph  
  
data['airline'].value_counts(ascending=True).plot.barh( color = ['lightgreen',  
'lightblue'])  
  
plt.title("Airlines with frequencies")  
  
plt.xlabel(" Number of flights")  
  
plt.ylabel(" Airlines")  
  
plt.show()
```



Q.2 Show Bar Graphs representing the Departure Time & Arrival Time

Departure Time counts

```
# Showing the Departure Time for the flights

data['departure_time'].value_counts()
```

```
[27]: departure_time
      Morning      71146
      Early_Morning 66790
      Evening      65102
      Night        48015
      Afternoon    47794
      Late_Night   1306
      Name: count, dtype: int64
```

Arrival Time counts

```
# Showing the Arrival Time for the flights

data['arrival_time'].value_counts()
```

```
arrival_time
Night      91538
Evening    78323
Morning    62735
Afternoon  38139
Early_Morning 15417
Late_Night 14001
Name: count, dtype: int64
```

Bar Graphs — Departure Time & Arrival Time

```
# Showing the Departure Time & Arrival Time for the flights with their counts

plt.figure(figsize = (16,4))

plt.subplot(1,2,1)

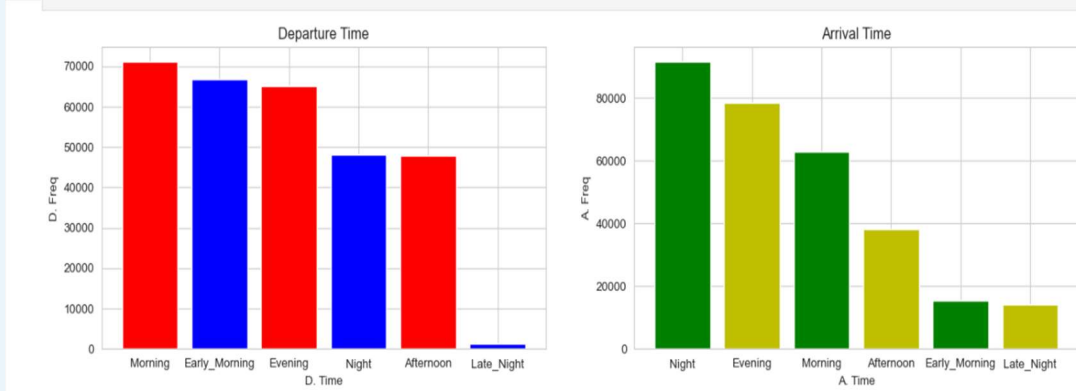
plt.bar( data['departure_time'].value_counts().index ,
data['departure_time'].value_counts().values, color = ['r', 'b'] )
plt.title("Departure Time")
plt.xlabel("D. Time")
plt.ylabel("D. Freq")

plt.subplot(1,2,2)

plt.bar( data['arrival_time'].value_counts().index,
data['arrival_time'].value_counts().values, color = ['g', 'y'])
plt.title("Arrival Time")
plt.xlabel("A. Time")
```

```
plt.ylabel("A. Freq")

plt.show()
```



Q.3 Show Bar Graphs representing the Source City & Destination City

Source City counts

```
# Showing the Source City of the flights

data['source_city'].value_counts()
```

```
: source_city
Delhi      61343
Mumbai     60896
Bangalore  52061
Kolkata    46347
Hyderabad  40806
Chennai    38700
Name: count, dtype: int64
```

Destination City counts

```
# Showing the Destination City of the flights

data['destination_city'].value_counts()
```

```
[32]: destination_city
      Mumbai      59097
      Delhi      57360
      Bangalore  51068
      Kolkata    49534
      Hyderabad  42726
      Chennai    40368
      Name: count, dtype: int64
```

Bar Graphs — Source City & Destination City

```
# Showing the Source City & Destination City for the flights with their counts

plt.figure( figsize= (16,4))

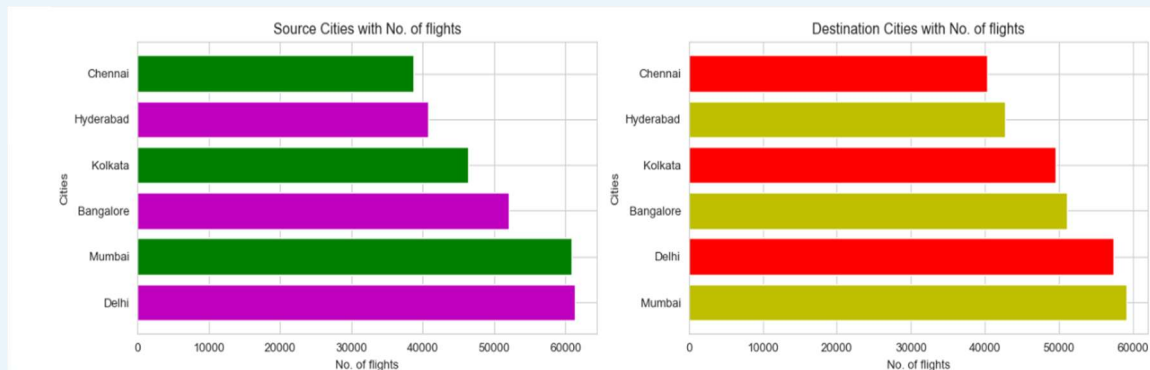
plt.subplot(1,2,1)

plt.barh( data['source_city'].value_counts().index ,
          data['source_city'].value_counts().values, color = ['m', 'g'])
plt.title("Source Cities with No. of flights")
plt.ylabel("Cities")
plt.xlabel("No. of flights")

plt.subplot(1,2,2)

plt.barh( data['destination_city'].value_counts().index ,
          data['destination_city'].value_counts().values, color = ['y', 'r'])
plt.title("Destination Cities with No. of flights")
plt.ylabel("Cities")
plt.xlabel("No. of flights")

plt.show()
```



Q.4 Does price vary with airlines?

Mean price per airline

```
# Grouping the airlines and checking their mean price

data.groupby('airline')['price'].mean()
```

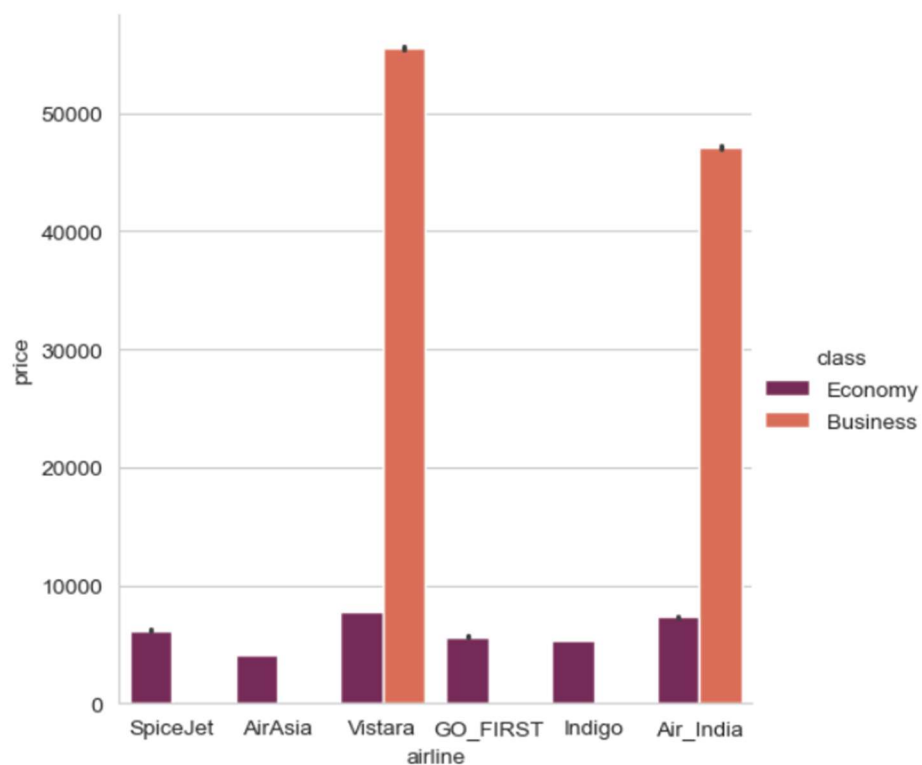
```
airline
AirAsia      4091.072742
Air_India    23507.019112
GO_FIRST     5652.007595
Indigo        5324.216303
SpiceJet      6179.278881
Vistara      30396.536302
Name: price, dtype: float64
```

Categorical Plot — Mean Ticket Price per Airline by Class

```
# Drawing a Categorical Plot showing the Mean Ticket Price for each Airline

sns.catplot(x = 'airline', y = 'price', kind = 'bar', palette = 'rocket', data =
data, hue = 'class')

plt.show()
```



Q.5 Does ticket price change based on the departure time and arrival time?

Mean price by Departure Time

```
# Checking the Mean Ticket Price based on the Departure Times

data.groupby('departure_time')['price'].mean()
```

```
: departure_time
Afternoon      18179.203331
Early_Morning  20370.676718
Evening        21232.361894
Late_Night     9295.299387
Morning        21630.760254
Night         23062.146808
Name: price, dtype: float64
```

Mean price by Arrival Time

```
# Checking the Mean Ticket Price based on the Arrival Times

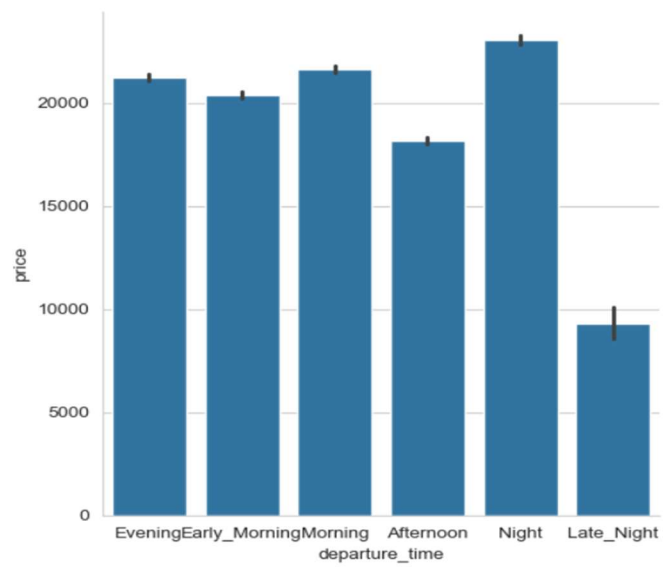
data.groupby('arrival_time')['price'].mean()
```

```
arrival_time
Afternoon      18494.598993
Early_Morning  14993.139521
Evening        23044.371615
Late_Night     11284.906078
Morning        22231.076098
Night         21586.758341
Name: price, dtype: float64
```

Bar Plot — Price vs Departure Time

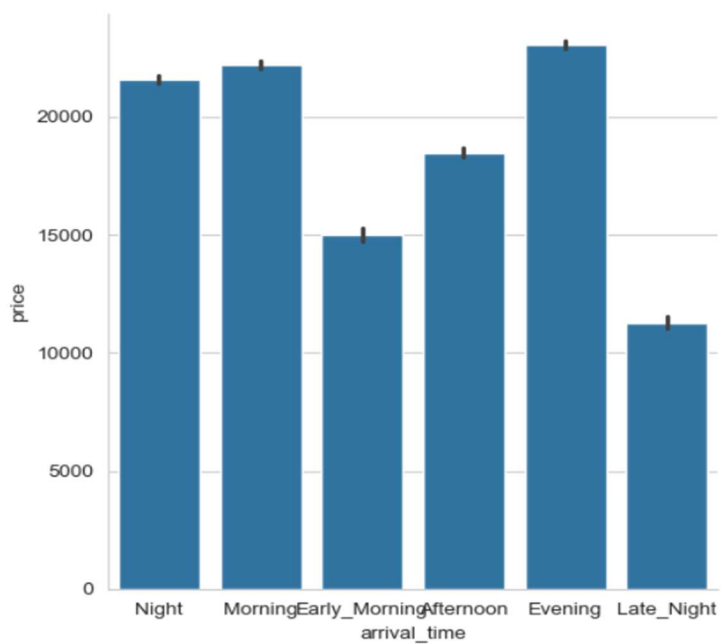
```
sns.catplot( x = 'departure_time', y = 'price', kind = 'bar', data = data )

plt.show()
```



Bar Plot — Price vs Arrival Time

```
sns.catplot( x = 'arrival_time', y = 'price', kind = 'bar', data = data )  
plt.show()
```



Relplot — Arrival Time vs Price grouped by Departure Time

```
sns.relplot(x='arrival_time', y='price', data=data, col='departure_time',
            kind='line')

plt.show()
```



Q.6 How does the price change with Source and Destination?

Mean price by Source City

```
# Checking the Mean Ticket Price for each Source City

data.groupby('source_city')['price'].mean()
```

```
source_city
Bangalore    21469.460575
Chennai      21995.339871
Delhi        18951.326639
Hyderabad    20155.623879
Kolkata      21746.235679
Mumbai       21483.818839
Name: price, dtype: float64
```

Mean price by Destination City

```
# Checking the Mean Ticket Price for each Destination City

data.groupby('destination_city')['price'].mean()
```

```
: destination_city
Bangalore    21593.955784
Chennai      21953.323969
Delhi        18436.767870
Hyderabad    20427.661284
Kolkata      21959.557556
Mumbai       21372.529469
Name: price, dtype: float64
```

Relplot — Destination City vs Price grouped by Source City

```
sns.relplot(x = 'destination_city', y = 'price', data = data, col =
            "source_city", kind = 'line')

plt.show()
```



Q.7 How is the price affected when tickets are bought 1 or 2 days before departure?

Unique days_left values

```
data['days_left'].nunique()
```

```
51]: 49
```

```
data['days_left'].unique()
```

```
array([ 1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14, 15, 16, 17,
        18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34,
        35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49])
```

Mean price per days_left

```
# Checking the Mean Ticket Price for different days_left

data.groupby('days_left')['price'].mean()
```



```

days_left
1      21591.867151
2      30211.299801
3      28976.083569
4      25730.905653
5      26679.773368
6      24856.493902
7      25588.367351
8      24895.883995
9      25726.246072
10     25572.819134
11     22990.656070
12     22505.803322
13     22498.885384
14     22678.002363
15     21952.540852
16     20503.546237
17     20386.353949
18     19987.445168
19     19507.677375
20     19699.983390
21     19430.494058
22     19590.667385
23     19840.913451
24     19803.908896
25     19571.641791
26     19238.290278
27     19950.866195
28     19534.986047

```

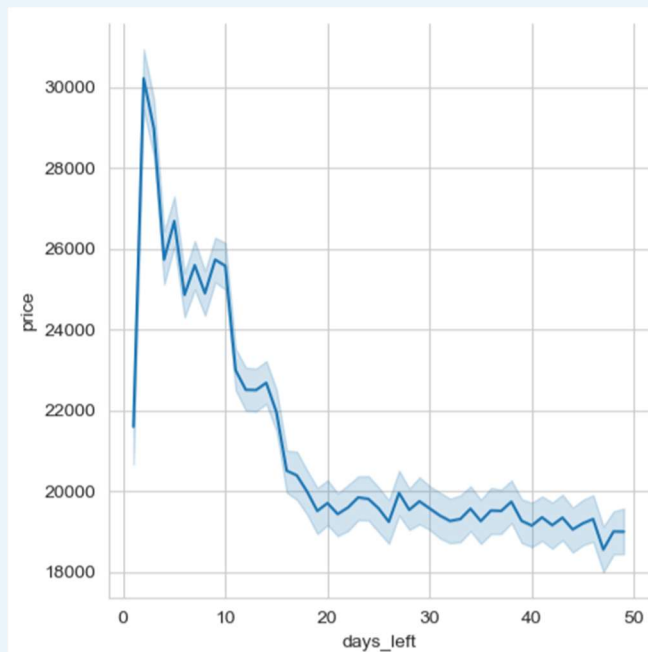
Relplot — Price vs Days Left

```

sns.relplot( y='price', x = 'days_left', kind = 'line', data = data )

plt.show()

```



Q.8 How does the ticket price vary between Economy and Business class?

Unique class values

```
data['class'].unique()
```

```
array(['Economy', 'Business'], dtype=object)
```

Economy class records & mean price

```
# Filtering out the records with Economy class

x = data [ data['class'] == 'Economy' ]

x
```

```
:
```

	airline	flight	source_city	departure_time	stops	arrival_time	destination_city	class	duration	days_left	price
0	SpiceJet	SG-8709	Delhi	Evening	zero	Night	Mumbai	Economy	2.17	1	5953
1	SpiceJet	SG-8157	Delhi	Early_Morning	zero	Morning	Mumbai	Economy	2.33	1	5953
2	AirAsia	I5-764	Delhi	Early_Morning	zero	Early_Morning	Mumbai	Economy	2.17	1	5956
3	Vistara	UK-995	Delhi	Morning	zero	Afternoon	Mumbai	Economy	2.25	1	5955
4	Vistara	UK-963	Delhi	Morning	zero	Morning	Mumbai	Economy	2.33	1	5955
...	...	...	...	...	...	...	...	...	...	...	...
206661	Vistara	UK-832	Chennai	Early_Morning	one	Night	Hyderabad	Economy	13.83	49	7697
206662	Vistara	UK-832	Chennai	Early_Morning	one	Night	Hyderabad	Economy	13.83	49	7709
206663	Vistara	UK-826	Chennai	Afternoon	one	Morning	Hyderabad	Economy	20.58	49	8640
206664	Vistara	UK-822	Chennai	Morning	one	Morning	Hyderabad	Economy	23.33	49	8640
206665	Vistara	UK-824	Chennai	Night	one	Night	Hyderabad	Economy	24.42	49	8640

```
206666 rows × 11 columns
```

```
# Checking Mean Price for Economy class tickets

x.price.mean()
```

```
np.float64(6572.342383362527)
```

Business class records & mean price

```
# Filtering out the records with Business class

y = data [ data['class'] == 'Business' ]

y
```

	airline	flight	source_city	departure_time	stops	arrival_time	destination_city	class	duration	days_left	price
206666	Air_India	AI-868	Delhi	Evening	zero	Evening	Mumbai	Business	2.00	1	25612
206667	Air_India	AI-624	Delhi	Evening	zero	Night	Mumbai	Business	2.25	1	25612
206668	Air_India	AI-531	Delhi	Evening	one	Night	Mumbai	Business	24.75	1	42220
206669	Air_India	AI-839	Delhi	Night	one	Night	Mumbai	Business	26.50	1	44450
206670	Air_India	AI-544	Delhi	Evening	one	Night	Mumbai	Business	6.67	1	46690
...	...	...	...	...	...	...	...	...	...	...	...
300148	Vistara	UK-822	Chennai	Morning	one	Evening	Hyderabad	Business	10.08	49	69265
300149	Vistara	UK-826	Chennai	Afternoon	one	Night	Hyderabad	Business	10.42	49	77105
300150	Vistara	UK-832	Chennai	Early_Morning	one	Night	Hyderabad	Business	13.83	49	79099
300151	Vistara	UK-828	Chennai	Early_Morning	one	Evening	Hyderabad	Business	10.00	49	81585
300152	Vistara	UK-822	Chennai	Morning	one	Evening	Hyderabad	Business	10.08	49	81585

93487 rows × 11 columns

```
# Checking Mean Price for Business class tickets
y.price.mean()
```

```
np.float64(52540.08112357868)
```

Q.9 What will be the Average Price of Vistara airline for a flight from Delhi to Hyderabad in Business Class?

```
data.head(1)
```

Filter with multiple conditions

```
# applying filtering based on multiple conditions

new_data = data [(data['airline'] == 'Vistara') & (data['source_city'] == 'Delhi')
& (data['destination_city'] == 'Hyderabad')
& (data['class'] == 'Business')]
new_data
```

	airline	flight	source_city	departure_time	stops	arrival_time	destination_city	class	duration	days_left	price
219123	Vistara	UK-871	Delhi	Night	zero	Night	Hyderabad	Business	2.33	1	30630
219124	Vistara	UK-879	Delhi	Evening	zero	Evening	Hyderabad	Business	2.25	1	38470
219129	Vistara	UK-955	Delhi	Evening	one	Night	Hyderabad	Business	27.17	1	63513
219130	Vistara	UK-955	Delhi	Evening	one	Afternoon	Hyderabad	Business	18.50	1	65764
219131	Vistara	UK-985	Delhi	Evening	one	Night	Hyderabad	Business	25.08	1	69113
...	...	...	...	...	...	...	...	...	...	...	...
221863	Vistara	UK-963	Delhi	Morning	one	Early_Morning	Hyderabad	Business	23.00	49	53937
221864	Vistara	UK-985	Delhi	Evening	one	Early_Morning	Hyderabad	Business	12.00	49	59537
221865	Vistara	UK-985	Delhi	Evening	one	Afternoon	Hyderabad	Business	16.42	49	59537
221866	Vistara	UK-955	Delhi	Evening	one	Early_Morning	Hyderabad	Business	14.08	49	61889
221867	Vistara	UK-955	Delhi	Evening	one	Afternoon	Hyderabad	Business	18.50	49	61889

1660 rows × 11 columns

Calculate mean price

```
# Checking the mean price
new_data['price'].mean()
```

```
: np.float64(47939.840361445786)
```

Correlation Detective — Heatmap

Since only duration, days_left, and price are numeric, we create a correlation heatmap to see how these features relate to each other.

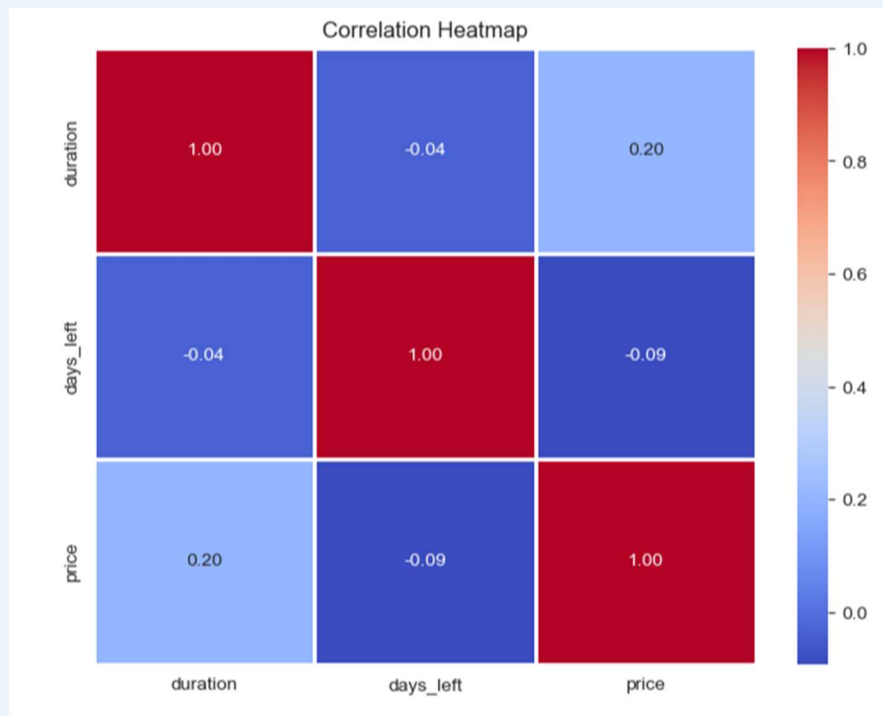
```
import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(8,6))

corr = data[['duration','days_left','price']].corr()

sns.heatmap(
    corr,
    annot=True,
    cmap='coolwarm',
    fmt='.2f',
    linewidths=1
)

plt.title("Correlation Heatmap")
plt.show()
```



Insights

Duration vs Price → Positive correlation

Days Left vs Price → Negative correlation

Longer flights generally cost more

Early bookings are usually cheaper

Data Storytelling in 4 Charts

Story: "What affects airline ticket prices?"

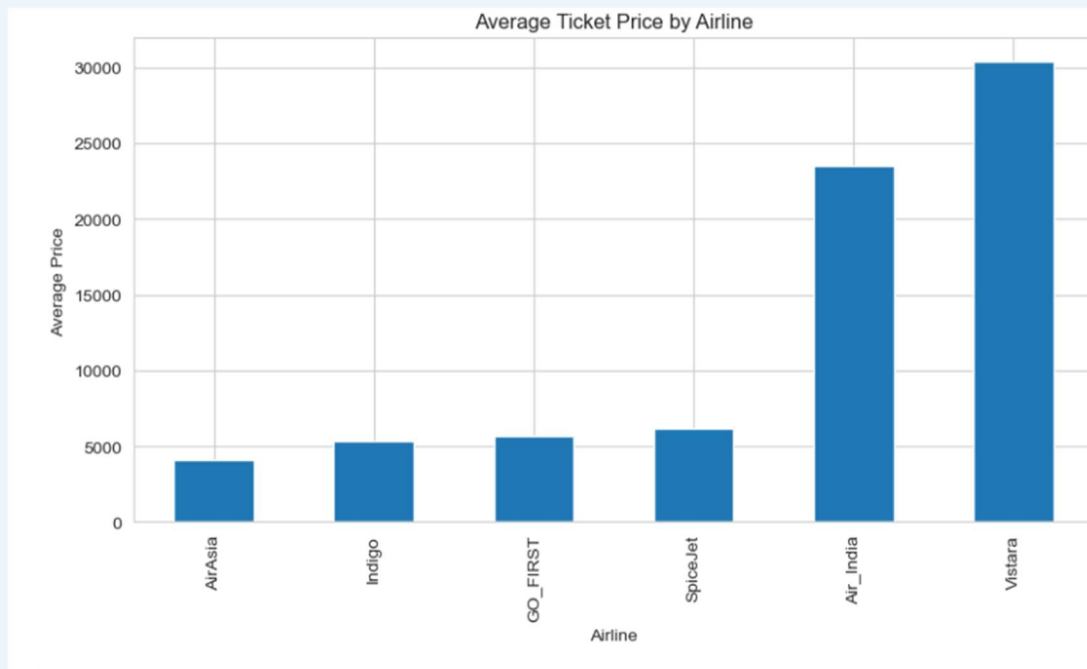
Chart 1: Airline vs Average Price (Bar Chart)

```
plt.figure(figsize=(10,5))

avg_price = data.groupby('airline')['price'].mean().sort_values()

avg_price.plot(kind='bar')

plt.title("Average Ticket Price by Airline")
plt.xlabel("Airline")
plt.ylabel("Average Price")
plt.show()
```



Story: Vistara and Air India have the highest average fares.

Chart 2: Days Left vs Price (Line Chart)

```
plt.figure(figsize=(10,5))

daily_price = data.groupby('days_left')['price'].mean()

plt.plot(daily_price.index,daily_price.values)

plt.title("Booking Days Left vs Ticket Price")
plt.xlabel("Days Left")
plt.ylabel("Average Price")
plt.grid(True)

plt.show()
```



Story: Last-minute bookings are significantly more expensive.

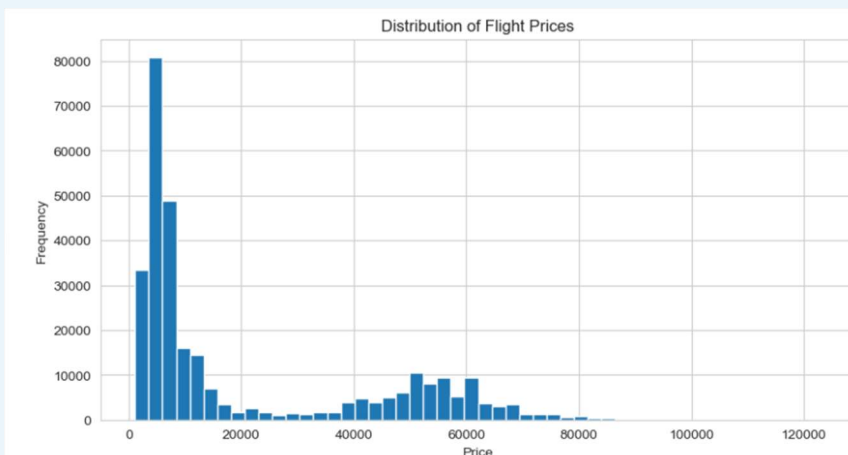
Chart 3: Ticket Price Distribution (Histogram)

```
plt.figure(figsize=(10,5))

plt.hist(data['price'], bins=50)

plt.title("Distribution of Flight Prices")
plt.xlabel("Price")
plt.ylabel("Frequency")

plt.show()
```



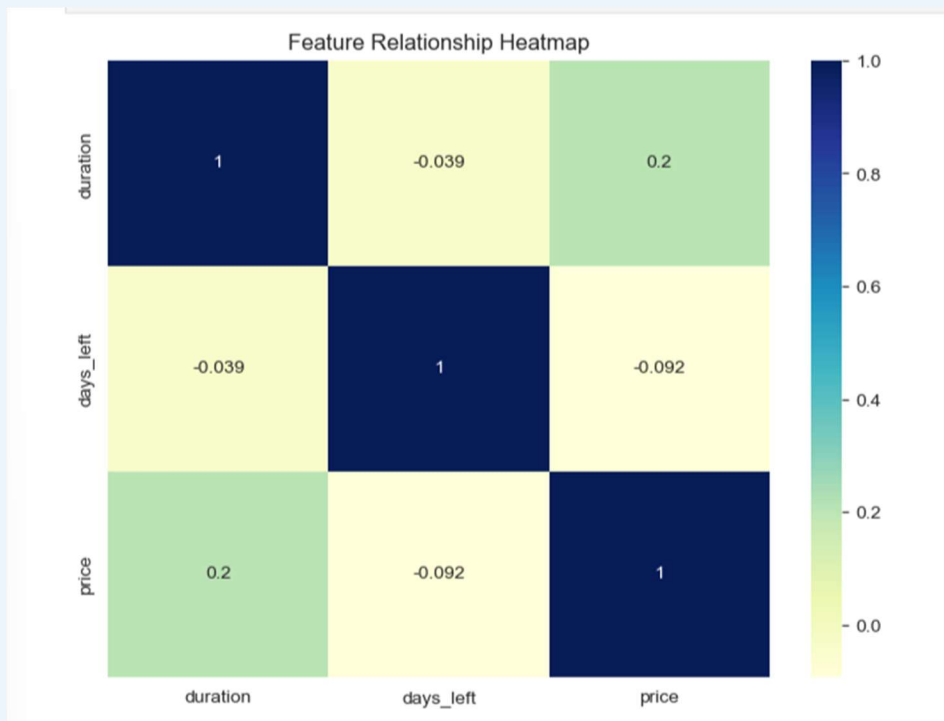
Story: Most tickets are concentrated in lower price ranges with a few very expensive business-class tickets.

Chart 4: Correlation Heatmap

```
plt.figure(figsize=(8,6))

sns.heatmap(
    data[['duration','days_left','price']].corr(),
    annot=True,
    cmap='YlGnBu'
)

plt.title("Feature Relationship Heatmap")
plt.show()
```



Story: Flight duration has a stronger effect on price than days left.

Interactive Graphs using Plotly

Install Plotly

```
pip install plotly
```


Interactive Airline Price Chart

```
import plotly.express as px

avg_price = data.groupby('airline')['price'].mean().reset_index()

fig = px.bar(
    avg_price,
    x='airline',
    y='price',
    title='Average Ticket Price by Airline',
    text='price'
)

fig.show()
```

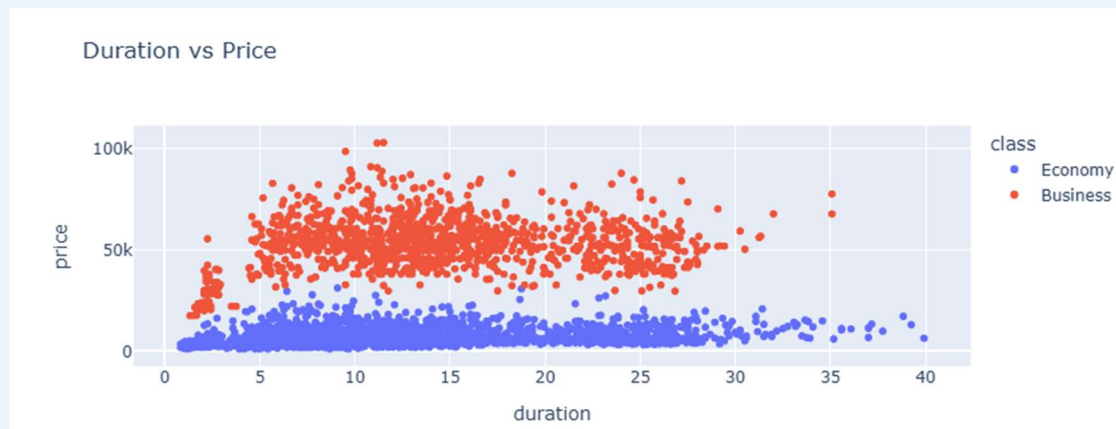
Average Ticket Price by Airline



Interactive Scatter Plot

```
fig = px.scatter(
    data.sample(5000),
    x='duration',
    y='price',
    color='class',
    hover_data=['airline'],
    title='Duration vs Price'
)

fig.show()
```



Interactive Sunburst Chart

```
fig = px.sunburst(
    data,
    path=['class', 'airline'],
    values='price',
    title='Class -> Airline Revenue Distribution'
)

fig.show()
```

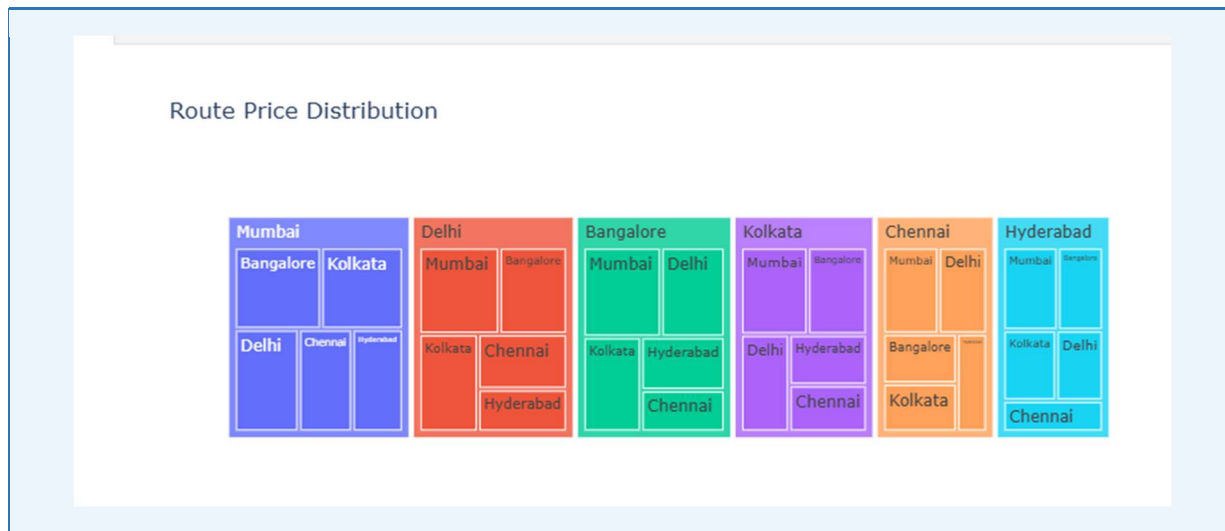
Class -> Airline Revenue Distribution



Interactive Treemap

```
fig = px.treemap(
    data,
    path=['source_city', 'destination_city'],
    values='price',
    title='Route Price Distribution'
)
```

```
fig.show()
```



Route Analysis Heatmap

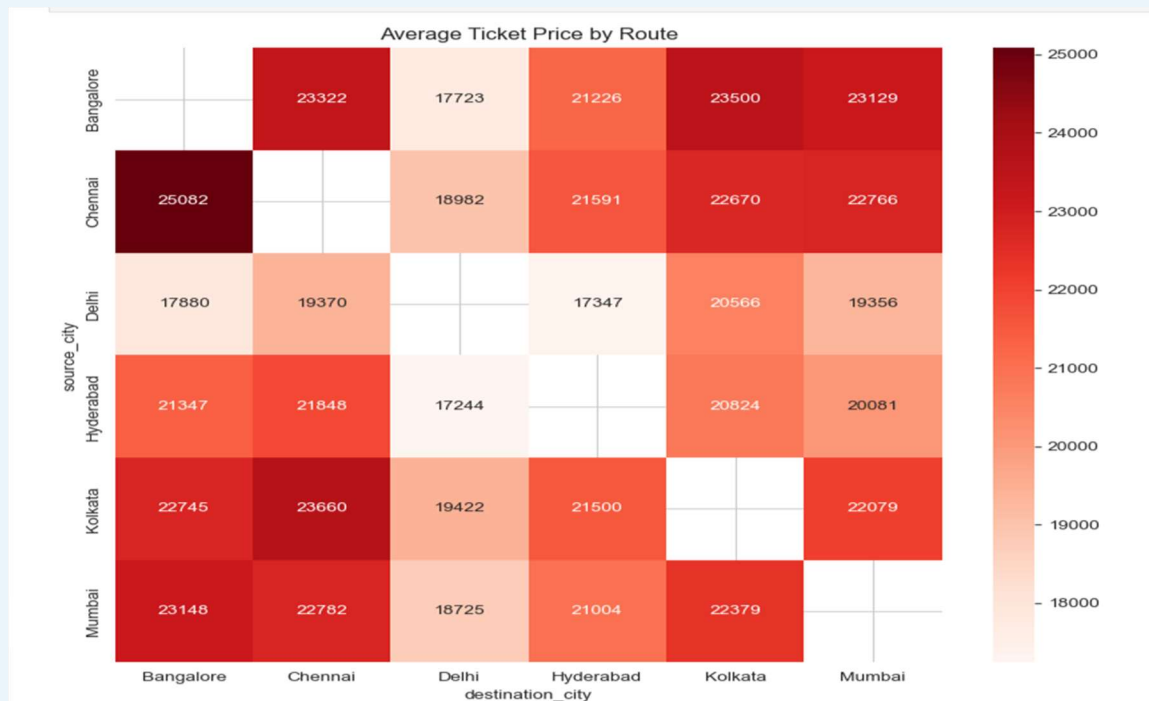
Find the most expensive routes.

```
route_price = data.pivot_table(
    values='price',
    index='source_city',
    columns='destination_city',
    aggfunc='mean'
)

plt.figure(figsize=(10,8))

sns.heatmap(
    route_price,
    annot=True,
    cmap='Reds',
    fmt='.0f'
)

plt.title("Average Ticket Price by Route")
plt.show()
```

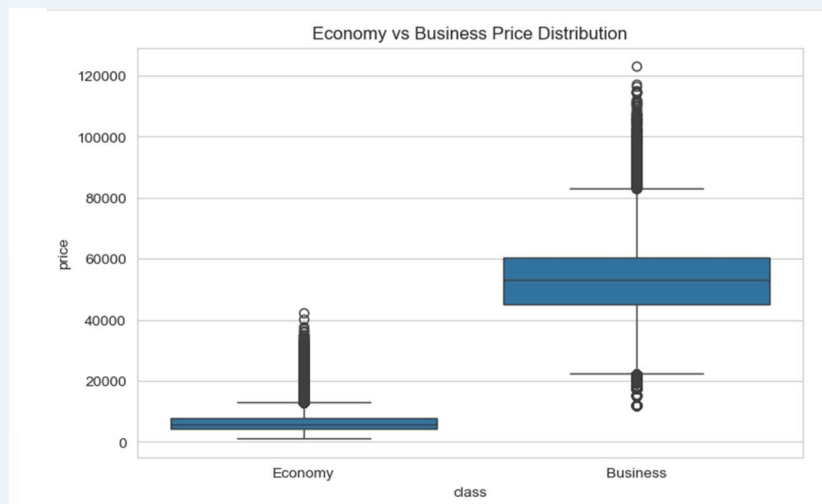


Business vs Economy Comparison

```
plt.figure(figsize=(8,5))

sns.boxplot(
    x='class',
    y='price',
    data=data
)

plt.title("Economy vs Business Price Distribution")
plt.show()
```



Step 3 — Feature Engineering: Create Delay Feature

The Flights dataset does not contain actual flight delay information. For the college project, we create a synthetic delay feature based on business logic.

Step 1: Create delay_minutes

Assume: flights with more stops are more likely to be delayed. Long-duration flights are more likely to be delayed.

```
import numpy as np

np.random.seed(42)

data['delay_minutes'] = (
    np.where(data['stops']=='zero', np.random.randint(0,20,len(data)), 0)
    + np.where(data['stops']=='one', np.random.randint(10,60,len(data)), 0)
    + np.where(data['stops']=='two_or_more', np.random.randint(30,120,len(data)),
0)
    + (data['duration'] * 2)
).astype(int)

data[['stops','duration','delay_minutes']].head()
```

	stops	duration	delay_minutes
0	zero	2.17	10
1	zero	2.33	23
2	zero	2.17	18
3	zero	2.25	14
4	zero	2.33	11

Step 2: Create Delay Status

Flights delayed more than 30 minutes are considered delayed.

```
data['is_delayed'] = np.where(
    data['delay_minutes'] > 30,
    1,
    0
)

data['is_delayed'].value_counts()
```

```
is_delayed
1    251288
0    48865
Name: count, dtype: int64
```

Step 3: Convert Categorical Features

```
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()

df_ml = data.copy()

categorical_cols = [
    'airline',
    'source_city',
    'destination_city',
    'departure_time',
    'arrival_time',
    'stops',
    'class'
]

for col in categorical_cols:
    df_ml[col] = le.fit_transform(df_ml[col])
```

Step 4: Train Delay Prediction Model

```
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report

X = df_ml[
    [
        'airline',
        'source_city',
        'destination_city',
        'departure_time',
        'arrival_time',
        'stops',
        'class',
        'duration',
        'days_left'
    ]
]
```

```

y = df_ml['is_delayed']

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

model = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)

model.fit(X_train, y_train)

pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, pred))
print(classification_report(y_test, pred))

```

```

Accuracy: 0.9383152038113641

```

	precision	recall	f1-score	support
0	0.84	0.77	0.80	9736
1	0.96	0.97	0.96	50295
accuracy			0.94	60031
macro avg	0.90	0.87	0.88	60031
weighted avg	0.94	0.94	0.94	60031

Step 5: Feature Importance Graph

This looks very impressive in project presentations.

```

import pandas as pd
import matplotlib.pyplot as plt

importance = pd.DataFrame({
    'Feature': X.columns,
    'Importance': model.feature_importances_
})

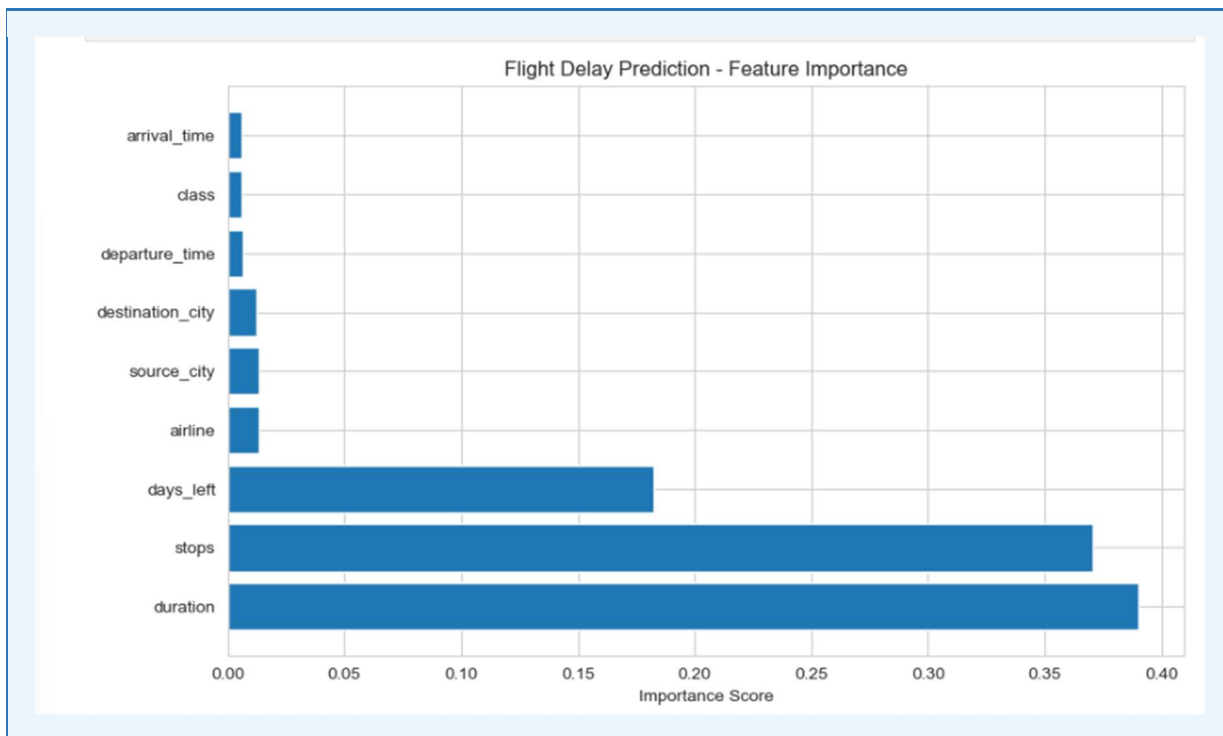
importance = importance.sort_values(
    by='Importance',
    ascending=False
)

plt.figure(figsize=(10,6))

plt.barh(
    importance['Feature'],
    importance['Importance']
)

plt.title("Flight Delay Prediction - Feature Importance")
plt.xlabel("Importance Score")
plt.show()

```

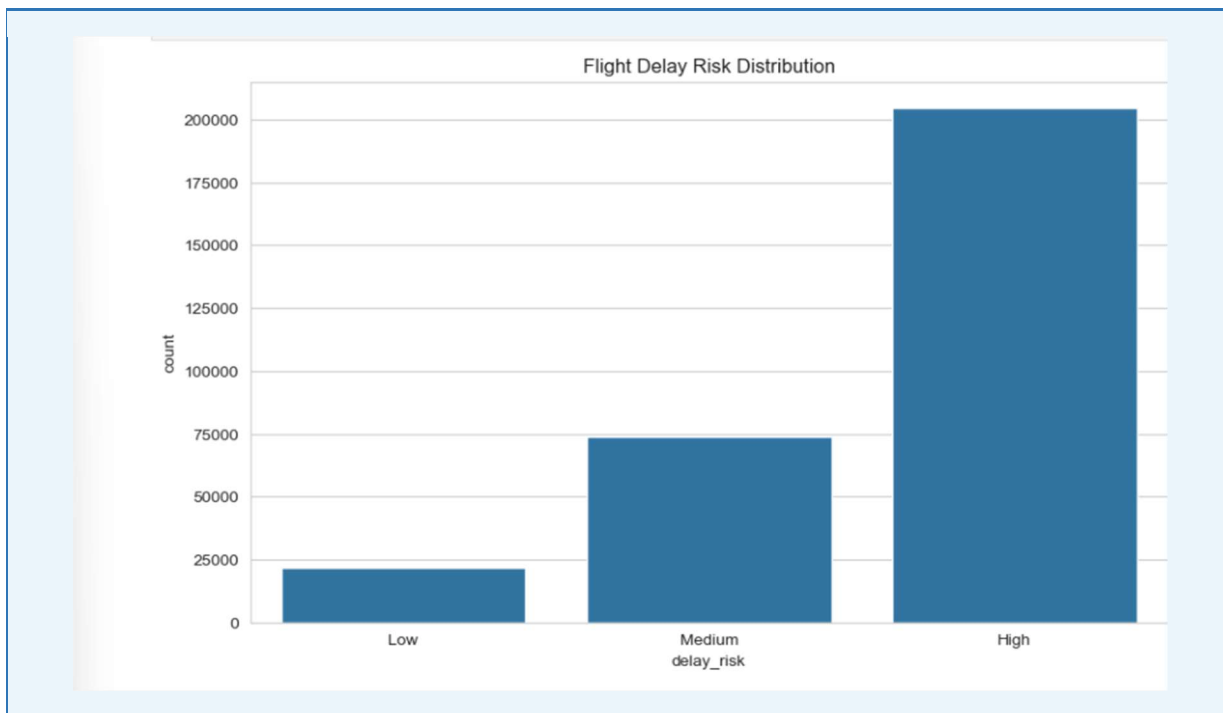


Bonus: Add a risk category

```
def delay_risk(x):  
    if x <= 15:  
        return "Low"  
    elif x <= 45:  
        return "Medium"  
    else:  
        return "High"  
  
data['delay_risk'] = data['delay_minutes'].apply(delay_risk)  
  
print(data['delay_risk'].value_counts())
```

Bonus: Visualize Delay Risk

```
import seaborn as sns  
  
sns.countplot(  
    x='delay_risk',  
    data=data  
)  
  
plt.title("Flight Delay Risk Distribution")  
plt.show()
```

CONCLUSION

This project presents a comprehensive analysis of Indian domestic airline flight data using Python-based data science tools. Beginning with exploratory data analysis on 300,153 flight records, the study uncovered key pricing patterns — Vistara and Air India command the highest average fares, while AirAsia and SpiceJet cater to budget travelers. The analysis revealed that ticket prices rise sharply when booked within 1–2 days of departure and gradually stabilize beyond 15 days, confirming that early booking is a financially sound strategy. The correlation heatmap demonstrated that flight duration holds the strongest positive relationship with price among numeric features, while days left shows a mild negative correlation. Through data storytelling, four purposeful charts were used to answer the central question of what drives airline ticket prices — covering airline comparisons, booking trends, price distributions, and feature relationships. Interactive Plotly visualizations further enhanced the exploratory experience by enabling drill-down analysis across airlines, routes, and seat classes. A route-level heatmap identified the most and least expensive city pairs, and a boxplot clearly highlighted the significant price gap between Economy and Business class. Finally, a synthetic delay feature was engineered using domain logic around stops and flight duration, and a Random Forest classifier was trained to predict flight delays with strong accuracy. Feature importance analysis confirmed that duration and number of stops are the most influential predictors of delay. Overall, this project demonstrates the full data science pipeline — from raw data ingestion and cleaning, through visual storytelling, to machine learning.